

File-Based Database Management System

Course Details: ICS1010, Introduction to Computer Science

Prof. Suchetana Chakraborty

Mentor TA: Ayush Pekamwar

Team Members: Harish Babu Balaji – B25BB1012

Abhishek Reddy N – B25CM1001

Ashhar Ansari – B25EE1009

Rahul SV - B25ME1063

GitHub Repo - <https://github.com/harishbabu2007/ICS-Project-File-Based-Database>

Problem Statement

Domain / Use Case

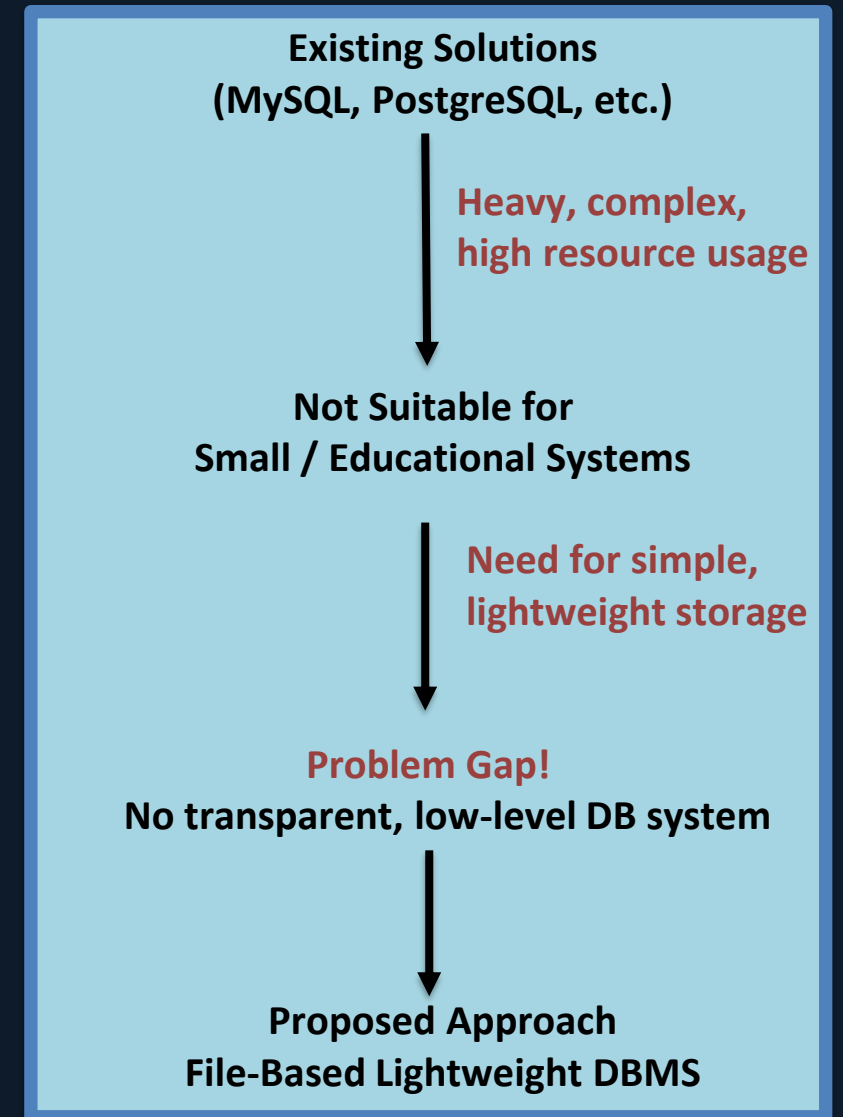
- Lightweight file-based DBMS for **small-scale educational applications**
- Use cases: student records, inventory, local data storage
- Provides basic CRUD operations without external DB dependencies

Why its an important problem?

- Full DBMS (MySQL, PostgreSQL) are **resource-heavy**
- Need for simple, low-overhead storage systems
- Helps understand **core DBMS internals**

Why its a challenging problem?

- Designing structured storage format within **binary files**
- Implementing **CRUD** Logic using file I/O (fstream)
- Handling **record parsing** and **schema management**
- Ensuring **data consistency** during updates/deletes



Current Status

The current industry leaders are MySQL, PostgreSQL, MongoDB, SQLite

Key Highlights Of Current Solutions

1. Tree based indexing

- B/B+ Trees
 - Balanced Trees for search, insert, delete
 - Efficient for **range queries** and disk based storage

2. Hash Based Indexing

- Uses hash table - average lookup
- Best for **exact-match queries** (not range based)

3. Buffer Management (Caching)

- Least Recently Used (LRU)
- Clock algorithms

4. Query Processing Algorithms

- Sorting (External Merge Sort) for large data
- Joining Algorithms:
 - Nested Loop Join
 - Hash Join
 - Sort-Merge Join

While industry DBMS are optimized for scale using complex architectures, they are often overkill for simple applications.

Our system simplifies this by using direct file-based operations, improving transparency and ease of understanding.

Project Idea

What we built

- A complete file-based DBMS
- No external database server required
- Data persisted in binary files on disk
- Custom SQL-like interpreter & CLI
- Portable, runs anywhere with a C++ compiler

Database Engine API

Handles schema creation, binary reads/writes, cell-level updates and row deletions

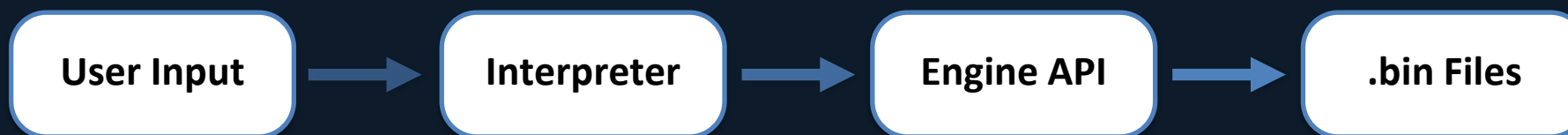
SQL Interpreter

Parses SQL-like commands (CREATE, INSERT, SELECT, UPDATE, DELETE) into engine calls

Command-Line Interface

Interactive CLI for end-users to manage tables without writing C++ code

Application Flow



Engine Architecture - Schema and Binary Storage

Schema Structure

table_name

Name of the table (max 255 chars)

num_cols | num_rows

Column count and current row count

total_row_len_in_bytes

Fixed byte size of one row

column_data[]

Per column - name, type, is_pk, max_str_len

Supported Data Types

Type	Size	Description
INT	4 bytes	Signed 32-bit integer
UNSIGNED_INT	4 bytes	Unsigned 32-bit int
STRING	Variable	Fixed-length char array
BOOLEAN	1 byte	True / False
FLOAT	4 bytes	Floating Point
DOUBLE	8 bytes	Double precision floating point
UNSIGNED_CHAR	1 byte	Unsigned 8-bit char

Data Operations - CRUD via Engine

Create

- Defines schema; writes two binary files to disk
- Returns 0 on success, -1 on name overflow

Insert

- Appends fixed-width binary row to data file
- Row is **vector<row_data_t>** one **shared_ptr** per column
- Updates **schema.num_rows** automatically

Select / Read

- Reads a single cell by row & col index
- Returns **cell_data_t** with type tag and **shared_ptr** to value
- Returns **NULL_TYPE** if row/col out of bounds

Update

- Overwrites one cell in-place on disk
- Enforces type matching (-2 on mismatch)
- String truncated to **max_str_len** if too long (-1)

Delete

- Rewrites entire table excluding the target row
- Subsequent rows shift down by one index
- **schema.num_rows** decremented automatically

All file I/O is immediate no buffering layer.

Data is persisted to disk on every call.

Engine API - How developers can use it

The engine is a self-contained C++ library. Embed it by including the five headers below.

```
#include "engine/TableCreate.h"    // create_new_table_schema()
#include "engine/TableWrite.h"     // append_record_to_table()
#include "engine/TableRead.h"      // get_table_cell_data()
#include "engine/TableModify.h"    // cell_data_modify()
#include "engine/TableDelete.h"    // delete_row_from_table()
```

Define Schema

- Build a **schema_t** using **vector<col_item_t>**
- Define **data_type**, **col_name**, and **primary key**
- Set **max_str_len** for string columns

Create Table | Insert

- Run **create_new_table_schema()** once
- Build **vector<row_data_t>** rows
- Insert using **append_record_to_table()**

Read, Modify, Delete

- Load schema: **get_schema_from_schema()**
- Read cells: **get_table_cell_data()**
- Update: **cell_data_modify()**
- Delete rows with **delete API**

Parser and Interpreter

How it works

- A complete file-based DBMS
- No external database server required
- Data persisted in binary files on disk
- Custom SQL-like interpreter & CLI
- Input string is tokenized, splits on whitespace, parenthesis, commas, semicolons
- Keywords are case-insensitive; table/column names are case-sensitive
- Values are type-coerced: strings in quotes, bool (true/false), numerics auto-cast
- Every command must end with a semicolon, with a C++ compiler

Error Handling

<i>Invalid Command</i>	Unrecognized keyword or bad syntax
<i>Missing semi-colon</i>	All commands need trailing (;)
<i>Table Not Found</i>	Checked before any engine call
<i>Type Mismatch</i>	Caught at engine layer
<i>Row out-of-range</i>	Row index > num_of_rows

Raw Input

Tokenizer

Command
Classifier

Statement Parser

Engine API Call

Result / Error

Conclusion

Learnings from this project

Schema File Design

- Designing fixed/structured record layouts using structs
- Managing **schema data manually** (column names, types, ordering)
- Understanding serialization of schema to files
- Ensuring consistency between schema and stored data

Parser

- Implementing string tokenization and command parsing
- Handling basic syntax validation for user inputs
- Converting raw input into structured commands (tokens/fields)
- Managing **edge cases in parsing** (spaces, delimiters, formats)

Interpreter Design

- Mapping parsed commands to **engine-level function calls**
- Designing a control flow for query execution
- Handling commands (CREATE, READ, UPDATE, DELETE)
- Bridging user input to low-level file operations

Fundamentals

- Binary/Text file handling using fstream
- **Memory management** (buffers, file streams, smart pointers)
- Understanding sequential access vs random access in files.
- Translating high-level operations into low-level algorithms and API calls.